Q5 – Derive a Generic Ranking Component from a Principle (7 Marks, K4)

> All numbers below were measured by me in a Java sandbox (2-core cloud machine, averaged over repeated runs after a warm-up). Absolute times will differ on your computer, so **re-run every program and report your own figures**; the trends and ratios are what matter. Predictions were derived from theory, but the assignment requires *you* to write your own prediction **before** you run each experiment, so write yours first. Each screenshot must be linked to the test it shows.

1. From the principle to a design

Principle: *a ranking is correct only if the comparison is consistent: if A ranks above B and B above C, then A above C; equal items compare as equal.* In Java terms, a comparison function `compare(a, b)` must satisfy:

1. **Antisymmetry:** `sign(compare(a,b)) = −sign(compare(b,a))`.
2. **Transitivity:** `compare(a,b) > 0` and `compare(b,c) > 0` ⇒ `compare(a,c) > 0`.
3. **Equality is consistent:** `compare(a,b) == 0` ⇒ `compare(a,c)` and `compare(b,c)` have the same sign for every c.
4. (Recommended) consistent with `equals`.

Sorting algorithms (TimSort) assume these rules; if they are violated the result can be wrong or an exception is thrown.

Design decisions

Question	Decision and reason
`Comparable<Team>` or separate `Comparator` objects?	**Separate `Comparators`.** `Comparable` gives a class exactly one "natural" order, but a team has several valid orders (league table, by name, by goal difference). The ranking component must be reused for players, colleges and quiz teams whose classes we may not control, and each of those would need to be changed to add `compareTo`. A comparator is supplied *from outside*, so the same `Leaderboard<T>` works for any type and the rule can change without touching the class.
How the multi-field comparison is expressed	`Comparator.comparingInt(points).reversed()` then `.thenComparing(goalDiff`

Question	Decision and reason
	`reversed)` then `.thenComparing(name)`: **lexicographic composition**: the next key is consulted only if all earlier keys tie.
Generic class that works for any rankable type	`class Leaderboard<T>` with a constructor parameter `Comparator<? super T>`. (If a type has a natural order, a bounded type `<T extends Comparable<? super T>>` offers a second constructor.)
Where is a wildcard useful	`Comparator<? super T>`: a comparator written for a *supertype* (e.g. `Comparator<Object>` or `Comparator<Person>`) can rank subtypes. `List<? extends T>/Collection<? extends T>` as an input parameter: `void addAll(Collection<? extends T> more)` accepts a `List<Student>` for a `Leaderboard<Person>` (a producer, "PECS").

```
// illustrative snippet (not part of the run): wildcard on input
void addAll(Collection<? extends T> more) { items.addAll(more); }
```

2. Complete the Code (TODO 1 and 2 filled in, executed)

```
import java.util.ArrayList;
import java.util.Comparator;
import java.util.List;

class Team {
    final String name; final int points; final int goalDiff;
    Team(String name, int points, int goalDiff) {
        this.name = name; this.points = points; this.goalDiff =
goalDiff;
    }
    @Override public String toString() {
        return name + "(" + points + "," + goalDiff + ")";
    }
}

class Leaderboard<T> {
    private final List<T> items = new ArrayList<>();
    private final Comparator<? super T> order;
```

```
    Leaderboard(Comparator<? super T> order) { this.order = order; }

    void add(T item) { items.add(item); }

    List<T> ranking() {
        // TODO 1 (completed): sorted COPY; 'items' is not modified
        List<T> copy = new ArrayList<>(items);
        copy.sort(order);
        return copy;
    }
}

public class LeaderboardDemo {
    public static void main(String[] args) {
        // TODO 2 (completed): points (high first), then goal
difference (high first), then name (A-Z)
        Comparator<Team> byRank = Comparator.comparingInt((Team t) ->
t.points).reversed()
                .thenComparing(Comparator.comparingInt((Team t) ->
t.goalDiff).reversed())
                .thenComparing(t -> t.name);

        Leaderboard<Team> board = new Leaderboard<>(byRank);
        board.add(new Team("Panthers", 9, 4));
        board.add(new Team("Eagles", 9, 6));
        board.add(new Team("Cobras", 6, 1));
        board.add(new Team("Bisons", 9, 4));
        System.out.println(board.ranking());
    }
}
```

Output (matches the expected output):

```
[Eagles(9,6), Bisons(9,4), Panthers(9,4), Cobras(6,1)]
```

Why Bisons is ranked above Panthers: both have 9 points and goal difference 4, so the first two keys tie and the third key (name, A→Z) decides: "Bisons" comes before "Panthers" alphabetically. The comparator therefore returns a negative value for (Bisons, Panthers), so Bisons ranks first among the two.

`ranking()` copies the list (`new ArrayList<>(items)`) before sorting so that the original insertion order inside the `Leaderboard` is not changed and the method can be called repeatedly; `List.sort(order)` is the same algorithm as `Collections.sort(list, order)`.

3. Test your design (`ComparatorTests.java`)

```
import java.util.*;

/** Data set 1: many ties at each level. Data set 2: deliberately
```

```
faulty comparators. */
public class ComparatorTests {
    record T(String name, int points, int gd) {}
    public static void main(String[] a) {
        Comparator<T> good =
Comparator.comparingInt(T::points).reversed().thenComparing(Comparator
.comparingInt(T::gd).reversed()).thenComparing(T::name);
        List<T> ties = new ArrayList<>(List.of(new T("Delta", 9, 4),
new T("Alpha", 9, 4), new T("Charlie", 9, 6), new T("Bravo", 6, 1),
new T("Echo", 6, 1), new T("Foxtrot", 6, 3), new T("Golf", 3, 0)));
        Collections.shuffle(ties, new Random(5)); ties.sort(good);
        System.out.println("Data set 1 (ties at points, goal
difference and name levels), correct comparator:\n  " + ties);

        // Faulty 1: subtraction overflow
        Comparator<T> sub = (x, y) -> y.points() - x.points();
        T big = new T("Big", Integer.MAX_VALUE, 0), neg = new T("Neg",
-5, 0);
        System.out.println("\nFaulty 1 (subtraction): compare(Big,
Neg) = " + sub.compare(big, neg) + "  compare(Neg, Big) = " +
sub.compare(neg, big) + "\n  The subtraction overflows, so the
comparator says Neg(-5 points) ranks ABOVE Big(2147483647 points):
wrong ranking, although the signs look 'consistent'.");
        // Faulty 2: ignores the tie-breaker
        Comparator<T> noTie =
Comparator.comparingInt(T::points).reversed();
        List<T> t2 = new ArrayList<>(List.of(new T("Panthers", 9, 4),
new T("Eagles", 9, 6), new T("Bisons", 9, 4)));
        List<T> r1 = new ArrayList<>(t2); r1.sort(noTie);
Collections.reverse(t2); List<T> r2 = new ArrayList<>(t2);
r2.sort(noTie);
        System.out.println("\nFaulty 2 (ignores goal difference and
name): input order A -> " + r1 + "\n
input order B -> " + r2 + "\n  Same teams, different 'ranking'
depending on input order (stable sort keeps arrival order for
ties).");
        // Faulty 3: not transitive -> TimSort may throw
        Comparator<Integer> cyc = (x, y) -> (x % 3 == (y + 1) % 3) ? -
1 : (x.equals(y) ? 0 : 1);
        Random r = new Random(1); List<Integer> nums = new
ArrayList<>(); for (int i = 0; i < 200; i++) nums.add(r.nextInt(30));
        try { nums.sort(cyc); System.out.println("\nFaulty 3 (non-
transitive): sort completed (no exception on this input) - result is
still not a valid order"); }
        catch (IllegalArgumentException e) { System.out.println("\
nFaulty 3 (non-transitive comparator): " + e.getMessage()); }
    }
}
```

Data set 1 – several ties at each level (7 teams; correct comparator; output):

`Charlie (9,6) , Alpha (9,4) , Delta (9,4) , Foxtrot (6,3) , Bravo (6,1) , Echo (6,1) , Golf (3,0)`

(points tie → goal difference decides (Charlie 6 above Alpha/Delta 4; Foxtrot 3 above Bravo/Echo 1); goal difference tie → name decides (Alpha < Delta, Bravo < Echo).) The result is the same whatever the input order, which I confirmed by shuffling the input first.

Data set 2 – deliberately broken comparators

Faulty comparator	Observation	What it reveals
Subtraction: `(x,y) -> y.points() - x.points()`	with `Integer.MAX_VALUE` and –5 points: `compare(Big, Neg)` = 2,147,483,644 and `compare(Neg, Big)` = –2,147,483,644 → the comparator ranks the team with **–5 points above the team with 2,147,483,647 points**	integer subtraction overflows, so the sign is wrong: it violates the principle even though the code looks natural. Use `Integer.compare(a, b)`.
Ignores the tie-breakers: only `points`	same three teams give `[Panthers, Eagles, Bisons]` for one input order and `[Bisons, Eagles, Panthers]` for the reversed input: **Eagles (9,6), who must be first, is in the middle**, and the output depends on the input order	equal items must be *ranked* equal only when *all* ranking criteria tie; a stable sort silently keeps arrival order, which hides the bug on some inputs
Non-transitive (cyclic rule on 200 random numbers)	`IllegalArgumentException: Comparison method violates its general contract!`	TimSort detects the broken transitivity at run time (not always, only when the data triggers it): a program that passes small tests can still fail in production

4. Correctness inquiry – why my comparator satisfies the principle

My comparator is $c = c_1 \triangleright c_2 \triangleright c_3$, where c_1 = points (descending), c_2 = goal difference (descending) and c_3 = name (ascending). Each c_i is built from `Integer.compare` / `String.compareTo` (reversed for descending), which are total preorders: each is consistent and transitive by definition of the integers and of dictionary order. The composition `a c b =`

$c_1(a,b)$ if $c_1(a,b) \neq 0$, otherwise $c_2(a,b)$ if $\neq 0$, otherwise $c_3(a,b)$ is the **lexicographic order** on the key tuple `(-points, -goalDiff, name)`. Reasoning:

- **Antisymmetry:** the first key that differs decides, and every c_i is antisymmetric, so `c(a,b) = -c(b,a)`.
- **Transitivity:** suppose a above b and b above c. Let i be the first key where a and b differ and j the first where b and c differ. If i < j: a and c agree with b on keys before i, differ from it at i in the same direction as a, so `a` is above `c` at key i. If j < i: symmetric, b's difference at j carries over to c vs a. If i = j: the key-i values are strictly ordered a < b < c, and transitivity of c_i gives a < c. In all cases `a` is above `c`.
- **Equal items compare as equal:** `c(a,b) = 0` only if *all three* keys tie, in which case a and b are interchangeable for ranking purposes, and then `c(a,x)` and `c(b,x)` always have the same sign. No subtraction is used, so no overflow is possible, and no key is skipped, so no tie is left unresolved (except genuinely identical team data).

5. Alternative concept and complexity analysis

(a) **`Team implements Comparable<Team>`** with the same chain inside `compareTo`:

```
class Team implements Comparable<Team> {
    public int compareTo(Team o) {
        int c = Integer.compare(o.points, points); if (c != 0) return
c;
        c = Integer.compare(o.goalDiff, goalDiff);  if (c != 0) return
c;
        return name.compareTo(o.name);
    }
}   // Collections.sort(list)  /  list.sort(null)
```

(b) Three stable sorts, from least to most important key: sort by name, then by goal difference, then by points (stability keeps the earlier order among ties). `SortStyles.java` timed (a)/(chain) against (b) and checked that both give the **same** result:

```
import java.util.*;

/** Alternative: Comparable<Team> vs 3 stable sorts (name, then goal
difference, then points). */
public class SortStyles {
    record T(String name, int points, int gd) {}
    public static void main(String[] a) {
        Comparator<T> chain =
Comparator.comparingInt(T::points).reversed().thenComparing(Comparator
.comparingInt(T::gd).reversed()).thenComparing(T::name);
        for (int n : new int[]{10_000, 100_000, 500_000}) {
            Random r = new Random(1); List<T> base = new
ArrayList<>(); for (int i = 0; i < n; i++) base.add(new T("Team" +
r.nextInt(n), r.nextInt(40), r.nextInt(30) - 15));
            for (int w = 0; w < 2; w++) { new
ArrayList<>(base).sort(chain); }
            double one = 0, three = 0; int reps = 3; boolean same =
```

```
true;
            for (int k = 0; k < reps; k++) {
                List<T> x = new ArrayList<>(base); long t0 =
System.nanoTime(); x.sort(chain); one += (System.nanoTime() - t0) /
1e6;
                List<T> y = new ArrayList<>(base); t0 =
System.nanoTime();
                y.sort(Comparator.comparing(T::name));
y.sort(Comparator.comparingInt(T::gd).reversed());
y.sort(Comparator.comparingInt(T::points).reversed());
                three += (System.nanoTime() - t0) / 1e6;
                same &= x.equals(y);
            }
            System.out.printf("n=%7d  one sort, comparator chain =
%8.1f ms   three stable sorts = %8.1f ms   same result: %b%n", n, one
/ reps, three / reps, same);
        }
    }
}
```

n teams	one sort with the comparator chain	three stable sorts	same result
10,000	3.0 ms	11.1 ms	yes
100,000	34.9 ms	71.4 ms	yes
500,000	364.3 ms	731.2 ms	yes

- **Time:** a sort is O(n log n). The chain is **one** O(n log n) sort whose comparison may check up to three keys; the three-sort approach does **three** O(n log n) sorts (total 3·n log n = still O(n log n)), measured 2–3.7× slower.
- **Space:** TimSort needs O(n) temporary space (up to n/2 references) for merging; the three-sort approach reuses the same list and does the allocation three times, so peak space stays O(n), but with three times the allocation.
- **Reuse for players and colleges:** the comparator version is easier. `Comparable` fixes one order inside the class, so ranking players *by a different rule* would need another class or a second mechanism; a `Comparator` is passed in. The three-sorts approach also relies on every step being stable and on remembering to apply the keys in reverse order, which is easy to get wrong. A `Comparator`-based `Leaderboard<T>` can be reused for any type by changing one argument.